TH-4: εκτέλεση του προγράμματος αυτοελέγχου

Ο 4-bit μικροελεγκτής σου (έκδοση 1.2, με timer) εκτελεί το `selftest.asm`, εντολή προς εντολή. Οι τιμές είναι ίδιες με αυτές που βγάζει η προσομοίωση (`vvp sim`).

1. Τι έχει μέσα ο TH-4

Κομμάτι	Μέγεθος	Τι κάνει
`A`	4 bits (0–15)	ο βασικός register· εδώ γίνονται οι πράξεις
`B`	4 bits (0–15)	δεύτερος register· μετρητής για βρόχους
`Z`	1 bit	flag: γίνεται 1 όταν ένα αποτέλεσμα είναι 0
`PC`	8 bits	η διεύθυνση της εντολής που εκτελείται
`RAM`	16 × 4 bits	16 θέσεις για μεταβλητές
`port_out`	4 bits	καταχωρητής της θύρας εξόδου (πόδια uio[7:4])
`port_in`	4 πόδια	η θύρα εισόδου (πόδια uio[3:0])
`timer`	25 bits	μετράει παλμούς ρολογιού μόνο όσο εκτελείται ένα WAIT· αλλιώς είναι σταματημένος, για οικονομία ρεύματος

Πώς είναι μια εντολή (1 byte)
`[7:4] = opcode  [3:0] = n`
Παράδειγμα: `ADD 3` = opcode 1, n 3 = `13` (hex).
Τα `JUMP`, `JZ`, `JNZ` πιάνουν **2 bytes**: το δεύτερο είναι η διεύθυνση.
Το opcode F είναι `TAB` όταν n = 0 και `WAIT n` όταν n = 1…15.

Στη δοκιμή, η θύρα εισόδου είναι σταθερή στο `1010` (= 10).

2. Οι εντολές

Hex	Εντολή	Τι κάνει	Αλλάζει το Z	Hex	Εντολή	Τι κάνει	Αλλάζει το Z
`0n`	`LOAD n`	A ← n	✔	`9n`	`LOADM n`	A ← RAM[n]	✔
`1n`	`ADD n`	A ← A + n	✔	`An`	`LDB n`	B ← n	
`2n`	`SUB n`	A ← A − n	✔	`B0`	`DECB`	B ← B − 1	✔
`3n`	`AND n`	A ← A AND n	✔	`C0 aa`	`JNZ aa`	αν Z = 0: PC ← aa	
`40`	`IN`	A ← θύρα εισόδου	✔	`Dn`	`ADDM n`	A ← A + RAM[n]	✔
`50`	`OUT`	θύρα εξόδου ← A		`En`	`SUBM n`	A ← A − RAM[n]	✔
`60 aa`	`JUMP aa`	PC ← aa (πάντα)		`F0`	`TAB`	B ← A	
`70 aa`	`JZ aa`	αν Z = 1: PC ← aa		`F1–FF`	`WAIT n`	περίμενε τον timer (n = 1…15)	
`8n`	`STORE n`	RAM[n] ← A					

3. Το πρόγραμμα στη μνήμη

Αριστερά η διεύθυνση και τα bytes που γράφονται στη μνήμη προγράμματος, δεξιά η assembly.

Διεύθ.	Bytes	Ετικέτα	Εντολή	Σχόλιο
00	05		LOAD 5	
01	13		ADD 3	A = 8
02	50		OUT	έξοδος = 8
03	28		SUB 8	A = 0, Z = 1
04	70 08		JZ ok1	πρέπει να πηδήξει
06	60 36		JUMP fail	
08	0C	ok1	LOAD 12	
09	36		AND 6	1100 & 0110 = 0100
0a	50		OUT	έξοδος = 4
0b	40		IN	θύρα εισόδου = 10
0c	50		OUT	έξοδος = 10
0d	83		STORE 3	RAM[3] = 10
0e	02		LOAD 2	
0f	87		STORE 7	RAM[7] = 2
10	93		LOADM 3	A = 10
11	D7		ADDM 7	A = 12
12	50		OUT	έξοδος = 12
13	E3		SUBM 3	A = 2
14	50		OUT	έξοδος = 2
15	E7		SUBM 7	A = 0, Z = 1
16	C0 36		JNZ fail	ΔΕΝ πρέπει να πηδήξει
18	A3		LDB 3	B = 3
19	00		LOAD 0	
1a	12	lp1	ADD 2	τρεις φορές -> A = 6
1b	B0		DECB	
1c	C0 1A		JNZ lp1	
1e	50		OUT	έξοδος = 6
1f	02		LOAD 2	
20	F0		TAB	B = 2
21	00		LOAD 0	
22	15	lp2	ADD 5	δύο φορές -> A = 10
23	B0		DECB	
24	C0 22		JNZ lp2	
26	50		OUT	έξοδος = 10
27	01		LOAD 1	Z = 0
28	70 36		JZ fail	ΔΕΝ πρέπει να πηδήξει

Διεύθ.	Bytes	Ετικέτα	Εντολή	Σχόλιο
2a	C0 2E		JNZ ok2	πρέπει να πηδήξει
2c	60 36		JUMP fail	
2e	07	**ok2**	LOAD 7	
2f	50		OUT	έξοδος = 7
30	F1		WAIT 1	περίμενε τον timer (1024 παλμούς)
31	F1		WAIT 1	και άλλους 1024
32	09		LOAD 9	
33	50		OUT	έξοδος = 9
34	60 34	**end**	JUMP end	
36	0F	**fail**	LOAD 15	
37	50		OUT	15 = κάτι πήγε στραβά
38	60 38	**stop**	JUMP stop	

4. Η εκτέλεση, βήμα προς βήμα

Κάθε γραμμή δείχνει τις τιμές **μετά** την εκτέλεση της εντολής.

πορτοκαλί η τιμή άλλαξε σε αυτό το βήμα κίτρινη γραμμή άλλαξε η θύρα εξόδου κόκκινο κείμενο έγινε άλμα

#	PC	Εντολή	A	B	Z	OUT	Τι έγινε
1	00	LOAD 5	**5**	0	0	0000	Βάζει τον αριθμό 5 στον A.
2	01	ADD 3	**8**	0	0	0000	A = 5 + 3 = 8.
3	02	OUT	8	0	0	**1000**	Η θύρα εξόδου παίρνει την τιμή του A: 8 = **1000**.
4	03	SUB 8	**0**	0	**1**	1000	A = 8 − 8 = 0. Το αποτέλεσμα είναι 0, άρα **Z = 1**.
5	04	JZ 08	0	0	1	1000	Z = 1, άρα **πηδάει** στη διεύθυνση 08.
6	08	LOAD 12	**12**	0	**0**	1000	Βάζει τον αριθμό 12 στον A. Δεν είναι 0, άρα Z = 0.
7	09	AND 6	**4**	0	0	1000	A = 1100 AND 0110 = 0100 (4). Μένουν 1 μόνο τα bits που είναι 1 και στα δύο.
8	0a	OUT	4	0	0	**0100**	Η θύρα εξόδου παίρνει την τιμή του A: 4 = **0100**.
9	0b	IN	**10**	0	0	0100	Διαβάζει τη θύρα εισόδου (1010) και τη βάζει στον A = 10.
10	0c	OUT	10	0	0	**1010**	Η θύρα εξόδου παίρνει την τιμή του A: 10 = **1010**.
11	0d	STORE 3	10	0	0	1010	Αποθηκεύει τον A (10) στη θέση 3 της RAM. Ο A δεν αλλάζει.
12	0e	LOAD 2	**2**	0	0	1010	Βάζει τον αριθμό 2 στον A.
13	0f	STORE 7	2	0	0	1010	Αποθηκεύει τον A (2) στη θέση 7 της RAM. Ο A δεν αλλάζει.
14	10	LOADM 3	**10**	0	0	1010	Φορτώνει στον A τη θέση 3 της RAM, που έχει 10.
15	11	ADDM 7	**12**	0	0	1010	A = 10 + RAM[7] = 10 + 2 = 12.
16	12	OUT	12	0	0	**1100**	Η θύρα εξόδου παίρνει την τιμή του A: 12 = **1100**.
17	13	SUBM 3	**2**	0	0	1100	A = 12 − RAM[3] = 12 − 10 = 2.
18	14	OUT	2	0	0	**0010**	Η θύρα εξόδου παίρνει την τιμή του A: 2 = **0010**.
19	15	SUBM 7	**0**	0	**1**	0010	A = 2 − RAM[7] = 2 − 2 = 0. Το αποτέλεσμα είναι 0, άρα **Z = 1**.
20	16	JNZ 36	0	0	1	0010	Z = 1, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (18).

#	PC	Εντολή	A	B	Z	OUT	Τι έγινε
21	18	`LDB 3`	0	**3**	1	0010	Βάζει τον αριθμό 3 στον B (μετρητής του βρόχου). Το Z δεν αλλάζει.
22	19	`LOAD 0`	0	3	1	0010	Βάζει τον αριθμό 0 στον A. Το αποτέλεσμα είναι 0, άρα **Z = 1**.
23	1a	`ADD 2`	**2**	3	**0**	0010	A = 0 + 2 = 2. Δεν είναι 0, άρα Z = 0.
24	1b	`DECB`	2	**2**	0	0010	B = 3 − 1 = 2. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
25	1c	`JNZ 1a`	2	2	0	0010	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 1a.
26	1a	`ADD 2`	**4**	2	0	0010	A = 2 + 2 = 4.
27	1b	`DECB`	4	**1**	0	0010	B = 2 − 1 = 1. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
28	1c	`JNZ 1a`	4	1	0	0010	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 1a.
29	1a	`ADD 2`	**6**	1	0	0010	A = 4 + 2 = 6.
30	1b	`DECB`	6	**0**	**1**	0010	B = 1 − 1 = 0. Ο B έφτασε στο 0, άρα **Z = 1**: ο βρόχος τελείωσε.
31	1c	`JNZ 1a`	6	0	1	0010	Z = 1, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (1e).
32	1e	`OUT`	6	0	1	**0110**	Η θύρα εξόδου παίρνει την τιμή του A: 6 = **0110**.
33	1f	`LOAD 2`	**2**	0	**0**	0110	Βάζει τον αριθμό 2 στον A. Δεν είναι 0, άρα Z = 0.
34	20	`TAB`	2	**2**	0	0110	Αντιγράφει τον A (2) στον B. Το Z δεν αλλάζει.
35	21	`LOAD 0`	**0**	2	**1**	0110	Βάζει τον αριθμό 0 στον A. Το αποτέλεσμα είναι 0, άρα **Z = 1**.
36	22	`ADD 5`	**5**	2	**0**	0110	A = 0 + 5 = 5. Δεν είναι 0, άρα Z = 0.
37	23	`DECB`	5	**1**	0	0110	B = 2 − 1 = 1. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
38	24	`JNZ 22`	5	1	0	0110	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 22.
39	22	`ADD 5`	**10**	1	0	0110	A = 5 + 5 = 10.
40	23	`DECB`	10	**0**	**1**	0110	B = 1 − 1 = 0. Ο B έφτασε στο 0, άρα **Z = 1**: ο βρόχος τελείωσε.
41	24	`JNZ 22`	10	0	1	0110	Z = 1, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (26).
42	26	`OUT`	10	0	1	**1010**	Η θύρα εξόδου παίρνει την τιμή του A: 10 = **1010**.
43	27	`LOAD 1`	**1**	0	**0**	1010	Βάζει τον αριθμό 1 στον A. Δεν είναι 0, άρα Z = 0.
44	28	`JZ 36`	1	0	0	1010	Z = 0, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (2a).
45	2a	`JNZ 2e`	1	0	0	1010	Z = 0, άρα **πηδάει** στη διεύθυνση 2e.
46	2e	`LOAD 7`	**7**	0	0	1010	Βάζει τον αριθμό 7 στον A.
47	2f	`OUT`	7	0	0	**0111**	Η θύρα εξόδου παίρνει την τιμή του A: 7 = **0111**.
48	30	`WAIT 1`	7	0	0	0111	Μηδενίζει τον timer και σταματάει το πρόγραμμα για 1.024 παλμούς ρολογιού, μέχρι να ανέβει το bit 10 του timer. Οι A, B, Z δεν αλλάζουν.
49	31	`WAIT 1`	7	0	0	0111	Μηδενίζει τον timer και σταματάει το πρόγραμμα για 1.024 παλμούς ρολογιού, μέχρι να ανέβει το bit 10 του timer. Οι A, B, Z δεν αλλάζουν.
50	32	`LOAD 9`	**9**	0	0	0111	Βάζει τον αριθμό 9 στον A.
51	33	`OUT`	9	0	0	**1001**	Η θύρα εξόδου παίρνει την τιμή του A: 9 = **1001**.
52	34	`JUMP 34`	9	0	0	1001	Πηδάει πάντα στη διεύθυνση 34 (στον εαυτό της: το πρόγραμμα «σταματάει» εδώ).

Αποτέλεσμα: η θύρα εξόδου πήρε με τη σειρά τις τιμές `8, 4, 10, 12, 2, 6, 10, 7, 9`. Αυτή είναι η σειρά που περιμένει ο πάγκος δοκιμών, γι' αυτό γράφει `ALL TESTS PASSED`. Αν κάποια εντολή δούλευε λάθος, το πρόγραμμα θα πήγαινε στην ετικέτα `fail` και η θύρα εξόδου θα έδειχνε 15.

5. Τι αξίζει να προσέξεις

- **Βήματα 3–5:** το `SUB 8` δίνει 0, το Z γίνεται 1 και το `JZ` πηδάει. Έτσι γίνεται ο έλεγχος «είναι ίσο με 8;».
- **Βήματα 11–20:** η RAM. Το `STORE` γράφει, τα `LOADM`, `ADDM`, `SUBM` διαβάζουν. Είναι οι μεταβλητές του προγράμματος.
- **Ο πρώτος βρόχος (`lp1`):** ο B ξεκινάει από 3 και κάθε γύρος κάνει `ADD 2`. Τρεις γύροι, άρα A = 6. Το `DECB` + `JNZ` είναι το `DEC` + `BRNE` του AVR.
- **Προσοχή στο Z μέσα στον βρόχο:** το `ADD` αλλάζει το Z, αλλά αμέσως μετά το `DECB` το ξαναγράφει. Το `JNZ` βλέπει πάντα το Z της **τελευταίας** εντολής που το άλλαξε.
- **Ο δεύτερος βρόχος (`lp2`):** ο B παίρνει τιμή από τον A με το `TAB`. Έτσι ο αριθμός των επαναλήψεων μπορεί να είναι αποτέλεσμα υπολογισμού.
- **Ο timer (βήματα 48–49):** κάθε `WAIT 1` φαίνεται ως μία γραμμή, αλλά το πρόγραμμα μένει εκεί για 1.026 παλμούς ρολογιού: 1.024 μετράει ο timer και 2 χρειάζεται η ίδια η εντολή. Ανάμεσα στην έξοδο 7 και την έξοδο 9 περνούν ακριβώς 2.054 παλμοί, ενώ όλες οι υπόλοιπες εντολές του προγράμματος μαζί χρειάζονται λιγότερους από 70.
- **Οικονομία ρεύματος:** ο timer μετράει μόνο μέσα σε ένα `WAIT`. Στα βήματα 1–47 είναι σταματημένος και τα flip-flops του δεν αλλάζουν τιμή. Κάθε `WAIT` τον μηδενίζει πρώτα, γι' αυτό κρατάει πάντα τον ίδιο χρόνο.